Synchronization-part2

Operating Systems

Universität Innsbruck

Summer Semester 2026

1 Deadlock: Incorrect Blocking

> **Problem:** After fixing race conditions with locks, threads may still stop making progress by waiting for each other in a circular way.
>
> **Question:** What happens after fixing race conditions, and why can the program still fail to finish?

The program no longer crashes and does not corrupt shared data. **However, it may still stop making progress.**

In this section, we study deadlock. Deadlock occurs when threads wait for resources held by each other, so none of them can proceed and the program never finishes.

> A program can be correct with respect to shared data and still never finish.
>
> This happens when threads wait on each other in a circular way.

1.1 Long Critical Section vs Small Critical Section

Before studying deadlock itself, it is useful to observe that bad lock usage can already make programs behave poorly.

Bad: holding a lock during slow work.

```
pthread_mutex_lock(&lock);
counter++;          // shared data access
sleep(2);           // slow work inside the critical section
pthread_mutex_unlock(&lock);
```

Better: protect only the shared state.

```
pthread_mutex_lock(&lock);
counter++;          // shared data access
pthread_mutex_unlock(&lock);
sleep(2);           // slow work moved outside
```

> A lock should protect only the shared data that needs synchronization.
>
> Slow work such as sleeping, I/O, or long computations should stay outside the critical section whenever possible.

1.2 Example: A Program That Freezes

The following POSIX threads program uses two mutexes. Each thread acquires the locks in a different order.

Code: 09_deadlock_example.c

```c
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t B = PTHREAD_MUTEX_INITIALIZER;

void* t1(void* arg) {
    printf("Thread 1: Locking A\n");
    pthread_mutex_lock(&A);

    sleep(1);

    printf("Thread 1: Locking B\n");
    pthread_mutex_lock(&B);

    printf("Thread 1: In critical section\n");

    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);
    printf("Thread 1: Released A and B\n");
    return NULL;
}

void* t2(void* arg) {
    printf("Thread 2: Locking B\n");
    pthread_mutex_lock(&B);

    sleep(1);

    printf("Thread 2: Locking A\n");
    pthread_mutex_lock(&A);

    printf("Thread 2: In critical section\n");

    pthread_mutex_unlock(&A);
    pthread_mutex_unlock(&B);
    printf("Thread 2: Released B and A\n");
    return NULL;
}

int main() {
    pthread_t thread1, thread2;

    pthread_create(&thread1, NULL, t1, NULL);
    pthread_create(&thread2, NULL, t2, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("If you see this line, no deadlock occurred\n");
    return 0;
```

```
53 }
```

Typical Output Before Freeze

```
Thread 1: Locking A
Thread 2: Locking B
Thread 1: Locking B
Thread 2: Locking A
```

> The program stops after this output due to a deadlock.
>
> Thread 1 holds mutex A and waits for B, while Thread 2 holds mutex B and waits for A, so neither can proceed.
>
> Deadlock is a progress problem, not a data-corruption problem.

1.3 What Happened

This program uses correct POSIX mutex operations: `pthread_mutex_lock`, `pthread_mutex_unlock`, `pthread_create`, `pthread_join`. The failure is not caused by an invalid API call. It is caused by the order in which locks are acquired.

1.4 Fix 1: Global Lock Ordering

A simple practical rule is to acquire multiple locks in the same order in every thread.

Ordered Pattern

```
1 pthread_mutex_lock(&A);
2 pthread_mutex_lock(&B);
3
4 /* critical section */
5
6 pthread_mutex_unlock(&B);
7 pthread_mutex_unlock(&A);
```

> If every thread locks A before B, circular wait cannot occur.

Corrected Code: `10_deadlock_fixed_order.c`

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER;
6 pthread_mutex_t B = PTHREAD_MUTEX_INITIALIZER;
7
8 void* t1(void* arg) {
9     printf("Thread 1: Locking A\n");
```

```
10      pthread_mutex_lock(&A);
11
12      sleep(1);
13
14      printf("Thread 1: Locking B\n");
15      pthread_mutex_lock(&B);
16
17      printf("Thread 1: In critical section\n");
18
19      pthread_mutex_unlock(&B);
20      pthread_mutex_unlock(&A);
21      printf("Thread 1: Released A and B\n");
22      return NULL;
23  }
24
25  void* t2(void* arg) {
26      printf("Thread 2: Locking A\n");
27      pthread_mutex_lock(&A);
28
29      sleep(1);
30
31      printf("Thread 2: Locking B\n");
32      pthread_mutex_lock(&B);
33
34      printf("Thread 2: In critical section\n");
35
36      pthread_mutex_unlock(&B);
37      pthread_mutex_unlock(&A);
38      printf("Thread 2: Released A and B\n");
39      return NULL;
40  }
41
42  int main() {
43      pthread_t thread1, thread2;
44
45      pthread_create(&thread1, NULL, t1, NULL);
46      pthread_create(&thread2, NULL, t2, NULL);
47
48      pthread_join(thread1, NULL);
49      pthread_join(thread2, NULL);
50
51      printf("Program finished normally\n");
52      return 0;
53  }
```

Output

```
Thread 1: Locking A
Thread 2: Locking A
Thread 1: Locking B
Thread 1: In critical section
Thread 1: Released A and B
```

```
Thread 2: Locking B
Thread 2: In critical section
Thread 2: Released A and B
Program finished normally
```

1.5 Fix 2: Acquire-All-or-Release

Instead of blocking on the second lock, a thread can attempt to acquire it without waiting.

Basic POSIX Usage

```c
pthread_mutex_lock(&A);

if (pthread_mutex_trylock(&B) == 0) {
    /* acquired both locks */
    pthread_mutex_unlock(&B);
} else {
    /* second lock not available */
}
pthread_mutex_unlock(&A);
```

> `pthread_mutex_trylock` attempts to acquire a mutex without blocking.
>
> If the mutex is already locked, the function returns immediately instead of waiting.

Modification in `11_1_deadlock_trylock.c`

Thread 1

```c
// Original
printf("Thread 1: Locking A\n");
pthread_mutex_lock(&A);

sleep(1);

printf("Thread 1: Locking B\n");
pthread_mutex_lock(&B);

// Replace with
while (1) {
    printf("Thread 1: Locking A\n");
    pthread_mutex_lock(&A);

    printf("Thread 1: Trying B\n");
    if (pthread_mutex_trylock(&B) == 0)
        break;

    printf("Thread 1: Could not get B, releasing A\n");
    pthread_mutex_unlock(&A);
    usleep(100000);
}
```

Thread 2

```c
// Original
printf("Thread 2: Locking A\n");
pthread_mutex_lock(&A);

sleep(1);

printf("Thread 2: Locking B\n");
pthread_mutex_lock(&B);

// Replace with
while (1) {
    printf("Thread 2: Locking B\n");
    pthread_mutex_lock(&B);

    printf("Thread 2: Trying A\n");
    if (pthread_mutex_trylock(&A) == 0)
        break;

    printf("Thread 2: Could not get A, releasing B\n");
    pthread_mutex_unlock(&B);
    usleep(100000);
}
```

Output

```
Thread 1: Locking A
Thread 1: Trying B
Thread 1: In critical section
Thread 1: Released A and B
Thread 2: Locking B
Thread 2: Trying A
Thread 2: In critical section
Thread 2: Released B and A
Program finished normally
```

> This version avoids deadlock by ensuring that a thread never waits while holding a lock.
>
> If it cannot acquire both locks, it releases the first one and retries.
>
> In practice, a fixed lock ordering is usually simpler and more predictable.

Practice Task (`11_1_deadlock_trylock.c`)

- Apply the `pthread_mutex_trylock` modification to both threads.
- Add a small delay using `usleep(1000)` before attempting the second lock to increase contention.
- Run the program several times and observe whether it terminates or gets stuck.

2 Producer–Consumer

> **Problem:** In the previous section, threads waited incorrectly and the program stopped making progress. This shows that not all waiting between threads is safe.
>
> **Question:** How can threads wait in a controlled way so that the program continues to run and eventually finishes?

The producer–consumer problem demonstrates how waiting can be used correctly for coordination.

A producer generates data and places it into a shared buffer. A consumer removes data from the buffer and processes it.

Unlike the deadlock case, threads do not wait indefinitely. They wait only when they cannot proceed:

- the producer waits if the buffer is full
- the consumer waits if the buffer is empty

As soon as the condition changes, waiting threads are resumed and the program continues.

2.1 Basic POSIX Usage

```c
sem_t sem;

sem_init(&sem, 0, value);

sem_wait(&sem);   // wait until resource is available
/* critical section */
sem_post(&sem);   // signal that resource is available

sem_destroy(&sem);
```

> `sem_wait` blocks when the condition is not satisfied.
>
> `sem_post` signals that another thread can continue.
>
> Semaphores were introduced earlier; here they are used for coordination.

2.2 Example: `12_prodcons_semaphores.c`

```c
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>

#define N 5
#define ITEMS 6

int buffer[N], in = 0, out = 0;
```

```c
sem_t empty_slots, full_slots;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;

void* producer(void* arg) {
    for (int item = 1; item <= ITEMS; item++) {
        sem_wait(&empty_slots);

        pthread_mutex_lock(&m);
        buffer[in] = item;
        in = (in + 1) % N;
        printf("Produced %d\n", item);
        pthread_mutex_unlock(&m);

        sem_post(&full_slots);
    }
    return NULL;
}

void* consumer(void* arg) {
    for (int i = 0; i < ITEMS; i++) {
        sem_wait(&full_slots);

        pthread_mutex_lock(&m);
        int item = buffer[out];
        out = (out + 1) % N;
        printf("Consumed %d\n", item);
        pthread_mutex_unlock(&m);

        sem_post(&empty_slots);
    }
    return NULL;
}

int main() {
    pthread_t p, c;

    sem_init(&empty_slots, 0, N);
    sem_init(&full_slots, 0, 0);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    return 0;
}
```

Typical Output

```
Produced 1
Produced 2
```

```
Produced 3
Consumed 1
Consumed 2
Consumed 3
Produced 4
Produced 5
Produced 6
Consumed 4
Consumed 5
Consumed 6
```

> `empty_slots` tracks free space in the buffer.
>
> `full_slots` tracks available items.
>
> The mutex protects the shared buffer state during updates.

2.3 Practice Task

- Set `N = 1` and run the program. Observe how producer and consumer alternate.

3 Readers–Writers

> **Problem:** In the previous section, threads waited correctly to coordinate access to shared data. However, using a single mutex forces all threads to access the data one at a time, even when this is not necessary.
>
> **Question:** How can we allow safe concurrent access when operations do not interfere with each other, while still protecting modifications?

Threads may access the same data in different ways. Some threads only read the data, while others modify it. Reading does not change the data, so multiple threads can read it at the same time without causing problems. Writing, however, changes the data and must be done by only one thread at a time. If a single mutex is used for both reading and writing, all threads are forced to access the data one by one. This ensures correctness, but it is inefficient because readers block each other even though they do not interfere.

3.1 Example: Mutex-Based Access

```c
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
int shared_data = 0;

void* reader(void* arg) {
    pthread_mutex_lock(&lock);
    printf("Reader sees %d\n", shared_data);
    pthread_mutex_unlock(&lock);
    return NULL;
}
```

> Even though readers do not modify the data, they are forced to run one at a time.

3.2 Solution: Read-Write Locks

POSIX provides read-write locks that allow:

- multiple concurrent readers
- exclusive access for writers

3.3 Basic POSIX Usage

```c
pthread_rwlock_t rw = PTHREAD_RWLOCK_INITIALIZER;

pthread_rwlock_rdlock(&rw);
/* read shared data */
pthread_rwlock_unlock(&rw);

pthread_rwlock_wrlock(&rw);
/* write shared data */
pthread_rwlock_unlock(&rw);
```

- `pthread_rwlock_rdlock`: acquires the lock for reading; multiple readers can hold it at the same time.
- `pthread_rwlock_wrlock`: acquires the lock for writing; only one writer is allowed and no readers can hold the lock.
- `pthread_rwlock_unlock`: releases the lock after reading or writing.

3.4 Example: `13_rwlock_demo.c`

```c
#define _XOPEN_SOURCE 700
#include <pthread.h>
#include <stdio.h>

int shared_data = 0;
pthread_rwlock_t rwlock = PTHREAD_RWLOCK_INITIALIZER;

void* reader(void* arg) {
    pthread_rwlock_rdlock(&rwlock);
    printf("Reader %ld sees %d\n", (long)arg, shared_data);
    pthread_rwlock_unlock(&rwlock);
    return NULL;
}

void* writer(void* arg) {
    pthread_rwlock_wrlock(&rwlock);
    printf("Writer updates %d -> %d\n", shared_data, shared_data + 1);
    shared_data++;
```

```
19      pthread_rwlock_unlock(&rwlock);
20      return NULL;
21  }
22
23  int main() {
24      pthread_t r1, r2, w;
25
26      pthread_create(&r1, NULL, reader, (void*)1);
27      pthread_create(&r2, NULL, reader, (void*)2);
28      pthread_create(&w, NULL, writer, NULL);
29
30      pthread_join(r1, NULL);
31      pthread_join(r2, NULL);
32      pthread_join(w, NULL);
33
34      return 0;
35  }
```

Typical Output

```
Reader 1 sees 0
Reader 2 sees 0
Writer updates 0 -> 1
```

> Multiple readers can access the data at the same time.
>
> A writer must wait until no reader or writer is active.

3.5 Practice Task

- Add a second writer thread and run the program. Verify that two writers never execute at the same time.

4 Dining Philosophers

> **Problem:** In the previous examples, threads coordinated access to a single resource. When multiple resources are required at the same time, even correct locking can lead to deadlock.
>
> **Question:** How can we safely coordinate threads that need to acquire multiple resources without causing deadlock?

In the producer–consumer problem, threads accessed a single shared buffer. In contrast, this problem involves multiple resources, and each thread must acquire more than one resource at the same time.

Each philosopher represents a thread, and each chopstick represents a resource. To perform its task, a philosopher must acquire two chopsticks.

> With multiple resources, the risk of circular waiting increases, making deadlock possible even when locks are used correctly.

4.1 Example: 15_philosophers_deadlock.c

```c
#include <pthread.h>
#include <stdio.h>

#define N 5
pthread_mutex_t chopstick[N];

void* philosopher(void* arg) {
    int id = (long)arg;

    pthread_mutex_lock(&chopstick[id]);
    // usleep(1000);  // force overlap
    pthread_mutex_lock(&chopstick[(id + 1) % N]);

    printf("Philosopher %d eating\n", id);

    pthread_mutex_unlock(&chopstick[(id + 1) % N]);
    pthread_mutex_unlock(&chopstick[id]);

    return NULL;
}

int main() {
    pthread_t t[N];

    for (int i = 0; i < N; i++)
        pthread_mutex_init(&chopstick[i], NULL);

    for (long i = 0; i < N; i++)
        pthread_create(&t[i], NULL, philosopher, (void*)i);

    for (int i = 0; i < N; i++)
        pthread_join(t[i], NULL);

    return 0;
}
```

Typical Output

```
Philosopher 0 eating
Philosopher 3 eating
Philosopher 4 eating
Philosopher 2 eating
Philosopher 1 eating
```

Each philosopher tries to acquire two resources.

If all threads pick up one resource first, a circular wait can occur and the program may freeze.

This code can deadlock, but the deadlock may not appear in every execution because it depends on scheduling.

Adding a small delay after the first lock, such as `usleep(1000)`, makes the deadlock easier to observe in practice.

4.2 Fix

In the deadlock section, two general solutions were introduced: using `pthread_mutex_trylock` and enforcing a consistent lock ordering. Those approaches are still applicable here.

In addition, this problem allows a simple specific solution: let one philosopher acquire the chopsticks in the reverse order.

By breaking the circular waiting pattern, deadlock is avoided.

Even a small change in ordering is enough to prevent all threads from waiting on each other.

5 Barrier Synchronization

Problem: In the previous examples, synchronization focused on accessing shared resources correctly. However, some problems require threads to coordinate their execution, not just their data access.

Question: How can we make threads wait for each other and continue only when all have reached the same point in execution?

So far, threads either competed for resources or coordinated access to shared data. In some problems, threads must wait for each other and proceed only after all have reached the same point.

A barrier is used when synchronization is about execution phases rather than resource access.

The example below uses the POSIX barrier functions `pthread_barrier_init`, `pthread_barrier_wait`, and `pthread_barrier_destroy`.

5.1 Basic POSIX Usage

```
1 pthread_barrier_t barrier;
2
3 pthread_barrier_init(&barrier, NULL, count);
```

```
4
5 pthread_barrier_wait(&barrier);   // wait until all threads arrive
6
7 pthread_barrier_destroy(&barrier);
```

- `pthread_barrier_init` initializes the barrier and sets how many threads must reach it before any can continue.
- `pthread_barrier_wait` blocks the calling thread until all required threads have reached the barrier, then all are released together.
- `pthread_barrier_destroy` releases resources after the barrier is no longer needed.

> A barrier synchronizes threads by forcing them to wait until all participants reach the same point in execution.

5.2 Example: `17_barrier_demo.c`

```c
#define _XOPEN_SOURCE 700
#include <pthread.h>
#include <stdio.h>

#define THREADS 4
pthread_barrier_t barrier;

void* worker(void* arg) {
    long id = (long)arg;

    printf("Thread %ld reached barrier\n", id);
    pthread_barrier_wait(&barrier);
    printf("Thread %ld passed barrier\n", id);

    return NULL;
}

int main() {
    pthread_t t[THREADS];

    pthread_barrier_init(&barrier, NULL, THREADS);

    for (long i = 0; i < THREADS; i++)
        pthread_create(&t[i], NULL, worker, (void*)i);

    for (int i = 0; i < THREADS; i++)
        pthread_join(t[i], NULL);

    pthread_barrier_destroy(&barrier);
    return 0;
}
```

Typical Output

```
Thread 1 reached barrier
Thread 0 reached barrier
Thread 3 reached barrier
Thread 2 reached barrier
Thread 2 passed barrier
Thread 1 passed barrier
Thread 0 passed barrier
Thread 3 passed barrier
```

> No thread continues past the barrier until all threads have reached it.

6 Thread-Safe Output

> **Problem:** In the previous examples, synchronization ensured correct access to shared data and prevented deadlock. However, concurrent execution can still produce confusing or inconsistent output.
>
> **Question:** How can we ensure that output from multiple threads remains clear and readable, even when threads run concurrently?

Even when synchronization correctly protects shared data and avoids deadlock, the program output may still be inconsistent.

> Correct synchronization ensures correctness of data, but it does not guarantee clean or readable output.
>
> Multiple threads can interleave their operations, leading to unexpected results.

6.1 Example: 18_output_interleaving.c

```c
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

void* worker(void* arg) {
    for (int i = 0; i < 5; i++) {
        printf("Thread %ld: part1 ", (long)arg);
        usleep(1000);   // force interleaving
        printf("part2 %d\n", i);
    }
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, worker, (void*)1);
```

```
18      pthread_create(&t2, NULL, worker, (void*)2);
19
20      pthread_join(t1, NULL);
21      pthread_join(t2, NULL);
22
23      return 0;
24  }
```

Possible Output

```
Thread 1: part1 Thread 2: part1 part2 0
Thread 2: part1 part2 0
Thread 1: part1 part2 1
Thread 2: part1 part2 1
Thread 1: part1 part2 2
Thread 2: part1 part2 2
Thread 1: part1 part2 3
Thread 2: part1 part2 3
Thread 1: part1 part2 4
part2 4
```

> The output becomes interleaved because multiple threads execute `printf` concurrently.
>
> Printing is not atomic, so parts of different lines can mix together.

6.2 Fix: Thread-Safe Output (`19_thread_safe_output.c`)

```
1   #include <pthread.h>
2   #include <stdio.h>
3
4   pthread_mutex_t print_lock = PTHREAD_MUTEX_INITIALIZER;
5
6   void safe_print(long id, int i) {
7       pthread_mutex_lock(&print_lock);
8       printf("Thread %ld: line %d\n", id, i);
9       pthread_mutex_unlock(&print_lock);
10  }
11
12  void* worker(void* arg) {
13      for (int i = 0; i < 5; i++) {
14          safe_print((long)arg, i);
15      }
16      return NULL;
17  }
18
19  int main() {
20      pthread_t t1, t2;
21
22      pthread_create(&t1, NULL, worker, (void*)1);
23      pthread_create(&t2, NULL, worker, (void*)2);
24
```

```
25      pthread_join(t1, NULL);
26      pthread_join(t2, NULL);
27
28      return 0;
29  }
```

Typical Output

```
Thread 2: line 0
Thread 2: line 1
Thread 2: line 2
Thread 2: line 3
Thread 2: line 4
Thread 1: line 0
Thread 1: line 1
Thread 1: line 2
Thread 1: line 3
Thread 1: line 4
```

> The mutex ensures that each print operation completes without interruption.
>
> Only one thread can print at a time, so lines remain complete and readable.

7 pthread_once

> **Problem:** In the previous examples, threads coordinated access to shared data and execution. However, some operations should not be repeated, even if multiple threads attempt to perform them.
>
> **Question:** How can we guarantee that a piece of code is executed exactly once, regardless of how many threads call it?

Not all synchronization problems involve coordination between many threads. Some tasks must be executed exactly once, even if multiple threads attempt them.

> Some synchronization problems are simple but critical: execute once and only once.

7.1 Basic POSIX Usage

```
1 pthread_once_t once = PTHREAD_ONCE_INIT;
2 void init();
3 pthread_once(&once, init);
```

- `pthread_once`: ensures that the initialization function is executed exactly once.
- `PTHREAD_ONCE_INIT`: initializes the control object.

7.2 Example (20_pthread_once_demo.c)

```c
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

pthread_once_t once = PTHREAD_ONCE_INIT;

/* Initialization function: runs exactly once */
void init_database() {
    printf("Connecting to database...\n");
    sleep(1);  // simulate expensive setup
    printf("Database initialized\n");
}

void* worker(void* arg) {
    long id = (long)arg;

    /* Ensure initialization happens only once */
    pthread_once(&once, init_database);

    printf("Thread %ld using database\n", id);
    return NULL;
}

int main() {
    pthread_t t1, t2, t3;

    pthread_create(&t1, NULL, worker, (void*)1);
    pthread_create(&t2, NULL, worker, (void*)2);
    pthread_create(&t3, NULL, worker, (void*)3);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);

    return 0;
}
```

Typical Output

```
Connecting to database...
Database initialized
Thread 1 using database
Thread 2 using database
Thread 3 using database
```

> The initialization function runs exactly once, even though all threads call pthread_once.
>
> Other threads wait until the initialization is complete, then continue normally.